

Part A – Easy

1. Stream Processing vs Batch Processing

Stream processing continuously processes data as it arrives, providing near real-time insights.

Batch processing, on the other hand, deals with finite datasets collected over a period and processes them all at once.

Example:

- **Batch:** Processing all day's transactions at midnight.
 - **Stream:** Continuously processing transactions as they occur to detect fraud instantly.
-

2. Challenges in Real-Time Stream Analytics

- Handling **unbounded data** and ensuring continuous computation.
 - Managing **out-of-order or late events**.
 - Maintaining **state consistency** in distributed systems.
 - Ensuring **fault tolerance** and **exactly-once guarantees**.
 - Balancing **latency vs throughput** trade-offs.
 - Integrating with **multiple sources/sinks** efficiently.
-

3. Stream, Event, Source, and Sink

- **Stream:** A continuous flow of data elements (e.g., sensor readings).
- **Event:** A single record in the stream (e.g., one sensor measurement).
- **Source:** Origin of the data stream (e.g., Kafka, socket, file).
- **Sink:** Destination where processed results are sent (e.g., database, file, dashboard).

Example: In a temperature monitoring system, sensor readings (events) flow from IoT sensors (source) into Flink and results are written to InfluxDB (sink).

4. Apache Flink Architecture (Core Components)

- **JobManager:** Coordinates the job execution, manages checkpoints, and task scheduling.
- **TaskManager:** Executes the actual tasks, holds local state, and reports status to JobManager.
- **Dispatcher:** Accepts job submissions and starts JobManagers.
- **ResourceManager:** Allocates cluster resources to TaskManagers.

Together, they form a **distributed, fault-tolerant stream processing system**.

5. Dataflow Programming in Flink

Flink programs are expressed as **dataflow graphs**, where nodes represent operations (map, reduce, etc.) and edges represent data streams.

This approach makes it easy to model computations as transformations over data streams.

6. Basic Dataflow Graph of a PyFlink Program

A typical PyFlink program includes:

1. **Source** → 2. **Transformations (map, flat_map, key_by, reduce)** → 3. **Sink**

Each transformation creates a new node in the dataflow graph.

(Sketch idea: Source → Map → KeyBy → Reduce → Sink)

7. Types of Time Semantics in Flink

- **Processing Time:** Based on the system's clock when the record is processed.
- **Event Time:** Based on timestamps in the event data itself.
- **Ingestion Time:** Based on the time Flink receives the record.

Example: In an IoT system, Event Time is when the reading was taken by the sensor.

8. Importance of Watermarks

Watermarks represent progress in event time and help handle late events by marking the point in time up to which all events are expected.

They ensure **event-time correctness** by allowing out-of-order events to be processed within tolerance limits.

9. KeyedStream vs DataStream

- **DataStream:** Represents an unkeyed stream where operations apply globally.
- **KeyedStream:** Partitioned stream based on keys, enabling keyed state and per-key computations.

Example: Counting clicks per user requires KeyedStream by user_id.

10. Parallelism in Stream Processing

Parallelism allows operators to execute across multiple tasks, increasing throughput.

Flink achieves distributed processing by splitting data streams and executing operators in parallel across multiple TaskManagers.

Part B – Medium (Applied Programming Tasks)

11. Convert Text File Data to Uppercase

```
from pyflink.datastream import StreamExecutionEnvironment

env = StreamExecutionEnvironment.get_execution_environment()
stream = env.read_text_file("input.txt")
```

```
upper_stream = stream.map(lambda x: x.upper())
upper_stream.print()
env.execute("Uppercase Transformation")
```

Explanation: Reads text from a file, converts each line to uppercase, and prints output.

12. Word Count using flat_map, key_by, reduce

```
from pyflink.datastream import StreamExecutionEnvironment

env = StreamExecutionEnvironment.get_execution_environment()
stream = env.read_text_file("input.txt")

words = stream.flat_map(lambda line: line.split(" ")) \
    .map(lambda word: (word, 1)) \
    .key_by(lambda x: x[0]) \
    .reduce(lambda a, b: (a[0], a[1] + b[1]))

words.print()
env.execute("Word Count")
```

13. WatermarkStrategy.for_monotonous_timestamps()

This strategy assumes timestamps are strictly increasing.

Flink generates watermarks equal to the current event timestamp.

Used when event timestamps are ordered.

```
from pyflink.common import WatermarkStrategy

stream = stream.assign_timestamps_and_watermarks(
    WatermarkStrategy.for_monotonous_timestamps()
    .with_timestamp_assigner(lambda e, ts: e[0])
)
```

14. Filter and Compute Average Sales per Store

```
from pyflink.datastream import StreamExecutionEnvironment
from pyflink.datastream.window import TumblingProcessingTimeWindows
from pyflink.common import Time
```

```
env = StreamExecutionEnvironment.get_execution_environment()
data = [(1000, "store1", 700), (2000, "store2", 300), (3000, "store1", 800)]
stream = env.from_collection(data)
```

```
filtered = stream.filter(lambda x: x[2] >= 500)
```

```
result = filtered.key_by(lambda x: x[1]) \
    .reduce(lambda a, b: (a[1], (a[2] + b[2]) / 2))
```

```
result.print()
```

```
env.execute("Average Sales Per Store")
```

15. Latency vs Throughput Trade-offs

- **Latency:** Time taken to process one event end-to-end.
- **Throughput:** Number of events processed per second.
Increasing throughput (batching, parallelism) often increases latency. Real-time systems must balance both depending on business needs.

16. Event-Time Semantics in Financial Systems

Benefits: Accurate handling of delayed trades, event-order consistency.

Drawbacks: Higher latency due to waiting for watermarks, complexity in managing lateness.

Part C – Hard (Analytical and Case-Based Questions)

17. IoT Sensor Pipeline Design

- **Use Event-Time semantics** for correctness.
- **Assign watermarks** to handle out-of-order events.
- **Enable checkpoints** for fault tolerance.
- **Maintain state** using KeyedStream with exactly-once guarantees.

```
env.enable_checkpointing(10000)
```

```
stream = env.from_source(iot_source, watermark_strategy, "IoT Source")
```

```
result = stream.key_by(lambda x: x.device_id) \
    .window(TumblingEventTimeWindows.of(Time.seconds(20))) \
    .reduce(lambda a, b: (a.device_id, (a.value + b.value)/2))
```

18. Data Exchange Strategies

Strategy	Description	When to Use	Impact
Forward	Sends data to the same parallel subtask	Data partition already aligned	Low overhead
Rebalance	Randomly redistributes elements	Balancing load across operators	May increase network I/O
Broadcast	Sends data to all downstream subtasks	Shared configuration or rules	High network cost

19. Alert for Active Users >1000

```
from pyflink.datastream.functions import KeyedProcessFunction
```

```
class UserAlert(KeyedProcessFunction):
```

```
    def process_element(self, value, ctx):
```

```
        if value[1] > 1000:
```

```
print(f"ALERT: {ctx.get_current_key()} exceeds 1000 users!")
```

Explanation: Each key (website) triggers an alert if active users exceed 1000 within 10 seconds.

20. Checkpoints vs Savepoints (Flink vs Spark)

Feature	Checkpoint	Savepoint	Spark Equivalent
Trigger	Automatic	Manual	Checkpoint in Spark
Purpose	Recovery	Upgrades / Migration	Checkpoint & WAL
Lifetime	Temporary	Persistent	RDD lineage
Guarantees	Exactly-once	Exactly-once	At-least-once (default)

21. Logistics Case Study (Truck GPS Monitoring)

Objective: Detect inactivity beyond 15 minutes.

Design:

- Use **Event Time** with **Watermarks**.
- Apply **Session Windows** to group active periods.
- Use **KeyedProcessFunction** to set a timer for each truck.
- Output inactivity alerts to a sink (e.g., Kafka).

```
class TruckMonitor(KeyedProcessFunction):
```

```
    def process_element(self, value, ctx):
```

```
        timer = ctx.timer_service().current_processing_time() + (15 * 60 * 1000)
```

```
        ctx.timer_service().register_processing_time_timer(timer)
```

```
    def on_timer(self, ts, ctx):
```

```
        print(f"Truck {ctx.get_current_key()} inactive for >15 minutes")
```